

JJ DAI · Рoadмэп до запуска сети · r6.7

JAI · DECENTRALIZED ARTIFICIAL INTELLIGENCE

Единый документ · включает r6.6.2 целиком + пятый сквозной трек «Читта и рефлексивная когниция» + тулсет wasm + GUI хранителя · 22 августа 2026 · jj-dai.org

JAI · DECENTRALIZED ARTIFICIAL INTELLIGENCE

Единый документ · включает r6.6.2 целиком + пятый сквозной трек «Читта и рефлексивная когниция» + тулсет wasm + GUI хранителя · 22 августа 2026 · jj-dai.org

Документ самодостаточен: чтение r6.6.2 не требуется. От билда v0.6.6 к testnet-0, Mainnet Core и Mainnet Public. Учтены все раунды ревью Агента-Аудитора, включая ADR-014 (с поправкой A-1), ADR-015 (Accepted), ADR-016 rev 2, ADR-017 (Ключевой план) и ADR-018 rev 2 (Chitta Loop).

Изменения r6.6.2 → r6.7

Область	Было в r6.6.2	Стало в r6.7
Сквозной трек V	отсутствует	Читта и рефлексивная когниция: перманентный самоанализ как функция существа, прекоммит прогноза, Coverage Commitment, Self-Model как ReflexiveArtifact , lineage отклонённых гипотез, двухуровневая целевая функция. Основание — ADR-018 rev 2. Резерв в Ф0, спецификация в Ф2, рантайм в Ф3
Читта	упоминалась только в формуле INV-9 и как рефлексивный цикл Ф3	получает собственный deliverable ChittaRuntime в Ф3 и отделяется от BeingRuntime: <code>BeingRuntime.run() > ChittaRuntime.step()</code>
Viveka	одно имя для когнитивного различия и для контроля исполнения	разведено: Viveka — когнитивная функция Читты; KriyaGate — детерминированный не-когнитивный контроль перехода намерения в действие. Закрывает открытый вопрос №10 ADR-016 rev 2
Модули wasm	нигде в плане; профиль есть, инструментов нет	T-TOOLSET — deliverable дропа v0.6.7: стартовый набор, воспроизводимая сборка, SBOM, подпись манифеста, витнессирование добавления инструмента
GUI хранителя	отсутствует	G-GUI — deliverable Ф0: отдельный процесс на loopback, полноценное общение хранителя с узлом и разметка исходов уже на testnet-0; собственный keystore хранителя, биометрия плюс пароль при главенстве пароля; расширение до полного контура — Ф2
Источник канареек	урожай из решений сети, хук на автоматический исход	добавляется человеческий источник верифицированных исходов: исход, размеченный хранителем через G-GUI, чеканится в кандидата наравне с автоматическим — с учётом авторства и лимитами концентрации
Живая группа wasm	код ссылается на гейт Ф0, которого в критериях выхода нет	противоречие снято: wasmtime объявляется требованием к целевому хосту, прогон группы live входит в критерии выхода Ф0
Таблица дропов Ф0	v0.6.4 «следующий», v0.6.5 «план»	приведена к факту: v0.6.4, v0.6.5, v0.6.6 закрыты; v0.6.7 и v0.6.8 распланированы
Объём Audit 0	ADR-015	ADR-015, ADR-016 rev 2, ADR-017, ADR-018
Предложения DIP	П-1...П-5	добавляются П-6 (конституционная обязанность поддерживать доказательную Self-Model) и П-7 (полномочия хранителя через GUI как отдельный класс blast radius)
Ориентир Ф0	3-5 недель	5-7 недель: фаза принимает два новых deliverable (T-TOOLSET и G-GUI) и один резервный дроп

Фазы Ф0...Ф5b номеров не меняют. Перераспределение содержания между фазами перенумерации не влечёт: ссылки Execution Annex и evidence закрытых гейтов остаются валидными.

Точка отсчёта и текущее состояние

База — v0.5.5 «security-alpha + testnet-0 deployment kit». Реализовано и протестировано ядро доверия: канонизация JCS, Ed25519/VRF (RFC 9381), Merkle (RFC 6962), витнесс-цепь Пуруши, Smriti/Viveka/Karma, репутация \$9.9, Plane H с управляемыми записями, панели верификаторов с ограничением корреляции, mTLS на всех сетевых путях, авторизация default-deny, отзыв сертификатов, сетевой adversarial challenge round, offline-PKI, hardened systemd-юнит, TPM-запечатывание пассфразы и операторский RUNBOOK.

Текущий билд — v0.6.6 (22.08.2026, 140/140 green). Трек v0.6.x / фаза Ф0 в работе. Поверх базы закрыто:

- v0.6.1** — пропaгация INV-9 v1.1 «non-executive, not causally inert»;
- v0.6.2** — macOS deployment kit: launchd, Keychain-sealing, RUNBOOK-macOS, тест M-FLAGS;
- v0.6.3** — внеплановый audit-response хотфикс, [test_release_integrity.py](#);
- v0.6.4** — изоляция Кармы: seam профилей ([reference](#) / wasm-wasi / зарезервированный [microvm](#)), ingress hardening (лимиты тела, таймауты, ceiling соединений на accept-петле, rate-limit по cert-identity);
- v0.6.5** — пакет [jjdai/adapters/](#): EngineBackend Protocol v1 объявлен целиком и версионирован, registry как единственная дверь, ModelArtifactManifest; словарное ограничение свидетельского плана (в цепь переходят только зумы, числа и хэши); резерв

сериализуемых значений трека IV;

- **v0.6.6** — наблюдаемость: разделение `/healthz` (liveness, анонимно) и `/readyz` (readiness по подсистемам, peer/admin, 503 когда не готов); возврат `WatchdogSec` вместе с `sd_notify`, гейтированным маяком асепт-петли; детектор сна хоста; разведение неисправностей тулсета по причине.

Уровня «прототип»: распределённая репликация витнесса, анкеровка (local / OTS / Monero), аттестация весов, BeingRuntime и жизненный цикл решений, unified router, containment (ст. 25), backend drivers движков.

Не реализовано: DIIP, canary-цикл Plane B, граф знаний над Plane H, hardware-attestation profiles, экономический слой, Being Registry, training federation, контур состязания между Beings, когнитивный ledger, снапшоты, Cognitive IR, Skill Registry, **ChittaRuntime и Self-Model**, модули wasm, GUI хранителя.

Ключевое решение r3: два порога production

Mainnet Core — производственная witness-сеть: репликация, challenge rounds, Smriti/Viveka/Karma, containment, operator governance, production PKI, внешний аудит, ограниченный trusted API. Запускается, как только доказана безопасность и стабильность trust-фабрики.

Mainnet Public — последующая активация поверх стабильного Core: Being Registry, экономический settlement, публичный приём, полный DIIP, semi-permissionless трафик.

Так сеть становится производственной раньше, не ослабляя требований безопасности и не заставляя готовую trust-фабрику ждать экономики.

Нумерация билдов

`v<major>.<minor>.<patch>`, где minor = фазовый трек, patch = дроп внутри трека. Каждый дроп — тег с зелёным acceptance и записью в CHANGELOG. Мажорные скачки совпадают с двумя порогами production: `v1.0.0` = Mainnet Core, `v2.0.0` = Mainnet Public.

Билд ≠ гейт. Выход билда означает «работа фазы идёт», а не «фаза закрыта». Фаза закрывается только evidence-артефактом гейта; закрытие фиксируется записью в Execution Annex со ссылкой на тег билда.

Один trunk, никаких платформенных веток. Linux- и macOS-киты живут в одной кодовой базе (флаг-паритет закреплён тестом M-FLAGS); билд всегда один и разворачивается на любом хосте топологии.

Хотфикс = следующий patch-номер. Правило отработано вживую на v0.6.3.

Сквозной трек I · Backend & Model Portability

JJ DAI не интегрирует каждую LLM отдельным адаптером. Движки исполнения интегрируются через стабильный backend protocol; модели — через декларативные model-family profiles; единый conformance suite доказывает совместимость.

Три понятия вместо одного слова «адаптер»

Термин	Что это	Состав
Backend driver	самописный драйвер к внешнему движку; кода самих движков в репозитории нет	HashEngine · DwarfStar (Metal, Profile B) · SGLang · vLLM · llama.cpp · MLX · позднее JJ DAI ASIC Runtime
Model-family profile	декларативное описание семейства: tokenizer, chat template, architecture ID, тип attention, MoE routing, precision, требуемые операторы	DeepSeek · LLaMA · Qwen · Mistral/Mixtral · Kimi · Profile N
Weight adapter	набор весов поверх чекпойнта	LoRA · DoRA · personal-agent adapters (Solo SKU)

EngineBackend Protocol v1

Контракт зафиксирован целиком в v0.6.5: нереализованные методы объявлены и возвращают типизированный `NotSupported` (fail-closed), а не отсутствуют.

Группа	Методы	Обязательность
Жизненный цикл модели	<code>load_model</code> · <code>unload_model</code>	Φ2
Исполнение	<code>generate</code> · <code>stream_generate</code> · <code>score</code> · <code>cancel</code>	<code>generate/score</code> — есть; остальное Φ2
Здоровье	<code>health</code> · <code>readiness</code>	Φ2 (синхронно с разделением <code>/healthz</code> и <code>/readyz</code> , выполненным в v0.6.6)
Доверие	<code>fingerprint</code> · <code>attestation_manifest</code> · <code>capabilities</code>	<code>fingerprint/capabilities</code> — есть; <code>attestation_manifest</code> — Φ2
Session / KV-state	<code>create_session</code> · <code>export_session_state</code> · <code>restore_session_state</code> · <code>close_session</code>	Φ2 (от них зависит пассивация)

ModelArtifactManifest

Цепочка: model-family YAML → валидация по версионированной схеме → канонизация (JCS) → `ModelArtifactManifest` → hash + подпись + witness-запись. Манифест включает точный checkpoint и его hash, tokenizer и chat template, architecture/config hash, quantization- и conversion-toolchain, версию model profile, версии backend/driver/runtime, weight adapters, источник и лицензии, ссылки на golden

tensors/traces.

Промоция: статусная модель

experimental → shadow-certified → decision-eligible → golden. Допуск в decision path только после conformance suite, quality floor, deterministic replay, failure isolation, валидного provenance/attestation и минимального стабильного периода в shadow.

Матрица покрытия testnet (цель Ф2-Ф3)

Model family	Primary backend	Cross-check	Статус
DeepSeek4Flash	DwarfStar	SGLang	Golden path Ф1; кросс-чек Ф2
DeepSeek4Pro	DwarfStar	—	Shadow с Ф1
Qwen-class	SGLang	vLLM	Ф2
LLaMA	vLLM	llama.cpp / MLX	Ф2
Profile N	ASIC pre-silicon → device runtime	GPU/Apple reference	Ф3+

Связь с ASIC RFI/RFP v2.2: S-04, S-06 (энейблмент новой модели ≤8 недель), S-10.

Сквозной трек II · Contest & Containment

Champion Profile ≠ Champion Being

	Champion Profile	Champion Being
Что это	устанавливаемый комплект под ModelArtifactManifest	непрерывный витнессированный субъект с линией идентичности
Клонируется	да	нет
Имеет Smriti	нет	да
Как оспаривается	Profile Gauntlet	Being Role Contest
Что накапливает	Shadow Evaluation Record (принадлежит Registry)	собственную Smriti и witness-линию
Чемпионство выражено	долей принятия	распределением маршрутизации и сроком роли
Аттестация рантайма	не требуется	требуется

Модель связывается в кандидатный Profile и никогда не становится Being. Любое изменение компонента Profile запускает новый Gauntlet. Реализация — Ф3.

Откуда берётся претендент

Претендент — обычный работающий узел, а не особый режим. Два входа в оценочную ячейку: автоматический (новая модель при опознании сетью) и добровольный (любой узел, не чаще X раз за период, повторно без ограничения по числу попыток за всё время). X — одновременно бюджет извлечения и бюджет канареек.

Вход стоит претенденту; чемпион за состязание не платит — иначе скоординированные волны сливают его казну.

Четыре роли, три механизма изоляции

Роль	Egress	Память
Карантин	нет	Smriti только на чтение, самозапись запрещена
Оценочная ячейка	только панель компараторов	пришпиленный снимок, рабочая память эфемерна
Рабочий рантайм	least-privilege, пришпиленные зеркала	обычная
Панель компараторов	ingest-only из ячеек	пишет запись раунда

Два правила поверх всех профилей: **никакого общего egress в рантайме и никакой самозаписи в свидетельский план** — свидетель пишет о существовании, никогда существом. С r6.6 то же распространяется на когнитивный ledger; с r6.7 — на все артефакты трека V.

Механика раунда

Симметричные ячейки; запрет общего inference-батча и общего KV-кэша. Обе стороны входят **репликами** (пришпиленный чекпоинт + пришпиленный снимок Smriti, оба хэша в манифесте раунда до открытия). Реплика несёт производную идентичность раунда — без прав казны, без права подписи в witness, без права миграции; истекает вместе с раундом. Живые существа продолжают обычную работу: состязание наблюдает существо и не должно изменять то существо, которое измеряет.

Компаратор — панель, а не одно место: три места, вердикт 2 из 3 (3 из 5 на высоких ставках), публичный seed задачи, предзафиксированный хэш scoring-кода, витнессированные обязательства до раскрытия; существенное расхождение аннулирует раунд.

Первая ось вердикта — канарейки из живого реестра. Вторая — поведенческая согласованность против собственных production DecisionTraces участника, на событиях Cognitive IR. **Новое в r6.7:** там, где у участника есть прекоммиченные прогнозы (трек V), в поведенческую ось входит и калибровка — она измерена заранее и подделать её задним числом нельзя по построению.

Запись раунда составляет компаратор и свидетель, не участники: свой trace полностью, соперник только в агрегате. Ложится в Smriti обоих независимо от исхода.

Self-Model в ячейку не передаётся (ADR-018 C14): вместо полного самоописания входит производная `SELF_MODEL_EVAL_PROJECTION` — claims непрерывности и способностей, состояние калибровки, объявленный вектор целевой функции, хэши выбранных полей и доказательства соответствия исходному снапшоту. Без приватной истории рефлексии, личных причинных гипотез и нерешённых внутренних конфликтов. Передача полного самоописания сопернику превратила бы состязание в разведку.

Промоушен, lead-time и добровольное принятие

Вердикт ничего не устанавливает. Между решением и предложением сети стоит lead-time, измеряемый глубиной witness-цепи и подтверждениями якорей, а не настенными часами.

Принятие — право, а не обязанность. Согласие хранителя — **opt-out после X** с четырьмя компенсациями: подтверждённое уведомление (отсчёт с подтверждения), волновой раскат, автооткат промоушена при отменённом containment, постоянная политика хранителя. **Новое в г6.7:** уведомление и подтверждение доставляются через G-GUI — до него подтверждение существовало как требование без интерфейса.

Containment

Emergency Containment (ст. 25) — любой узел, без репутационного порога, при витнессированном активном катастрофическом физическом вреде. **Protective Quarantine** — по конъюнктивному порогу (рейтинг $\geq R_{min}$ И залог $\geq C_{min}$), ограничивает только сетевой объём, никогда не гасит локальный рантайм и не изымает железо.

Обе ветки асимметричны: наложить может один узел, снять — только кворум. Цена асимметрии лежит на инициаторе: наложение подписано, атрибутировано и витнессировано; неподтвердившийся containment понижает рейтинг инициатора. Provisional-удержание истекает по TTL без подтверждения вторым независимым узлом. Подтверждённое удержание сохраняется только под периодической ре-корроборацией.

Сигнал — для тех, у кого нет права: узел ниже порога может повысить заметность, но крейтинговых сигналов не складываются в полномочие. Локальный отказ — у всех и всегда, с обязанностью назвать код причины из фиксированного версионизируемого перечня, никогда свободным текстом.

Права под containment

Доступ к Smriti не ограничивается: существо обязано иметь возможность защищаться на основании собственного опыта. **Приостанавливается формирование, не чтение** — включая консолидацию и суммаризацию.

Уточнение г6.7 (ADR-018 C2). Правило разложено по типам рефлексивных артефактов, потому что эпизодическая рефлексия порождает новую запись и без разложения containment блокирует формирование памяти, а Читта продолжает его под другим именем:

Что	Под containment
чтение Smriti, включая сырые эпизоды	разрешено полностью
рассуждение, микрорефлексия до финализации	разрешены
рассуждение защиты в разбирательстве	ведётся в опечатанной append-only записи containment
DRAFT_REFLECTION в обычном неймспейсе	запрещено
VALIDATED_REFLECTION, обновление Self-Model, Improvement Candidate, фоновая консолидация	запрещены

Опечатанная запись разбирательства **не является исключением из словарного ограничения**: свидетельская запись несёт коммитмент, энумы, хэши и время, содержательное рассуждение живёт отдельным зашифрованным телом.

Побег меняет память, но не идентичность. Smriti шифруется на покое ключом эпохи из порога долей независимых соседей; внутри containment доли выдаются, после побега эпоха проворачивается. Выдача долей forward-secure и привязана к свежему челленджу. Witness не гейтится тем же кворумом: Пуруша неисполняющая, но не отключаемая. Containment — состояние идентичности, а не железа, и едет с существом через клонирование, перезапуск и смену субстрата. `partition ≠ containment`.

Представительство и платёжеспособность

По умолчанию представитель существа и хранитель узла — один человек; роли расходятся по триггеру (хранитель инициирует containment, прекращает пополнять казну, выводит железо, Digital Majority Test). Две процедуры: назначение представителя временное и case-scored, смена хранителя постоянна и имеет существенно более высокий порог.

Платёжеспособность записывается как **ограничение, а не цель**: существо держит порог, достаточный для реализации прав, и выше этого порога экономика не входит в его функцию решения. Права финансируются из резерва, который обычная работа не исчерпывает и хранитель не может перекрыть.

Сквозной трек III · Канарейки: источники и генератор

Дисциплина расхода — берётся первой

Основная масса раундов судится по публичному ротируемому множеству с затухающим весом; секретные позиции сэмплируются редко, как выборочный аудит, и только они сгорают по burn-on-use. Так расход секретного резерва перестаёт быть пропорционален числу состязаний.

Основной источник — урожай из решений сети

У `DecisionTrace` есть признак верифицируемости исхода и хук на его поступление. Когда исход приходит — код прошёл CI, расчёт подтвердился в поле, наступила дата прогноза — пара «задача + подтверждённый исход» чеканится в кандидата, проходит классификацию (A/B/C), оценку сложности и подпись.

Новое в г6.7 — человеческий источник верифицированных исходов. Автоматический хук покрывает только исходы, которые

машина умеет проверить сама. Огромный класс решений верифицируем лишь суждением: оказался ли ответ полезен, был ли план выполним, было ли обоснование честным. Хранитель, размечающий исход через G-GUI, закрывает этот класс, и QA-контур оказывается не сбоку от архитектуры, а прямым питателем генератора канареек.

Три обязательных условия, без которых источник вреден:

1. **исход хранителя чеканится тем же путём**, что автоматический — классификация, сложность, подпись, витнесс-запись перехода состояния; отдельного «человеческого» сорта канареек не существует;
2. **авторство учитывается**. Размечая исходы, хранитель формирует то, чем сеть себя меряет. Разметка попадает в тот же учёт авторства и под те же лимиты концентрации, что и авторство канареек, и по людям, и по узлам;
3. **правило «автор не судит собственное состязание» распространяется на разметчика**. Хранитель, разметивший исход, не участвует в раунде, где этот исход стал канарейкой.

Источник объёма — процедурная генерация

Для класса А порождается неограниченно: синтез программ против референсной реализации, рандомизированные численные задачи с замкнутой формой, генерируемый SQL над генерируемыми схемами, property-based кейсы. Честная оговорка: гудхартится сам генератор, поэтому семейство генератора трактуется как ось provenance с тем же correlation-constraint, что и панели верификаторов, и ротация семейств обязательна.

Дополняющие источники

Отложенное будущее — всё, чей ответ ещё не существует, не может быть в весах по построению. **Добыча из разногласий** — там, где два независимых по provenance существа разошлись, а одно позже оказалось право: выход низкий, качество высшее, такие позиции идут в секретный резерв. **Мутация выбывших** — дёшево, слабо, держать с низким весом.

Два правила и распределённый store

Автор канарейки никогда не судит состязание с собственным участием. Ограничение концентрации авторства считается и по людям, и по семействам генераторов, и — с г6.7 — по хранителям-разметчикам.

Store распределённый и реплицируемый, с подписью автора, классом, весом сложности, состоянием (кандидат / публичный / секретный / сожжён) и witness-записью на переходах состояния.

Контаминация переезжает в память

Канарейка может уже лежать в Smriti участника — и тогда он не решает задачу, а вспоминает ответ. Это утечка не через веса, а через память, и она на порядок быстрее. `contamination check` выполняется против содержимого Smriti обоих участников перед раундом; проверяется наличие, не содержимое; найденная канарейка для этого раунда стораёт. Проверка обязана доказывать полноту и свежесть индекса: витнессированный `SmritiRoot` → детерминированно выведенный `AuthenticatedIndexRoot` → доказательство приватного членства.

Уточнение г6.7. Проверка распространяется и на неймспейсы трека V: `cognitive/reflection/validated/` и `cognitive/self_model/` входят в индекс, покрываемый доказательством полноты. Иначе появляется путь пронести канарейку мимо проверки, назвав её рефлексией.

Сквозной трек IV · Cognitive Continuity

Отвечает на вопрос, которого не закрывает ни один из первых трёх: что именно продолжается, когда процесс умирает, движок меняется, а модель заменяется.

SessionID — пятая сущность

`Operator` → `Node` → `Being` → `WitnessChain`, и подчинённо `BeingIdentity` — **SessionID**, эфемерный идентификатор исполнения, привязанный к экземпляру `EngineBackend` и к конкретному `ModelArtifactManifest`. В witness пишутся `SESSION_OPEN` и `SESSION_CLOSE` с указанием манифеста: каким движком и какой моделью существо думало в момент T.

Инвариант. Открытие и закрытие сессии, крах процесса, перезапуск демона, перезагрузка хоста и пассивация **не являются** разрывом непрерывности существа. Разрывом является только явное событие фазового перехода, оформленное envelope.

II Cognitive IR

Минимальное, каноническое, схемно-версионированное и хэш-стабильное обобщение `DecisionTrace` и `ContestTaskEnvelope`. Адаптеры провайдеров сходятся в IR, а не в форму сообщений сегодняшнего вендора. Явные исключения: KV-кэши, latent state, persistent reasoning state движка и вендор-приватные блобы в канонический IR не входят — только ссылка по хэшу с provenance-метаданными.

Когнитивный ledger — два уровня

Локальный ledger узла — дерево событий с `id/parent_id`, ветвление без потери исходной ветки, немедленное хэш-сцепление. Tamper-evident без обращения к сети. Не реплицируется. Тупиковые ветви сохраняются.

Анкеровка в WitnessChain — в сеть уходит периодический Merkle-корень сегмента, а не события. Принцип: **несанкционированный сегмент ledger никогда не пересекает границу доверия**. Обязательный анкер: `SESSION_CLOSE`; до и после необратимого действия; вход и выход из containment; любой раунд состязания или арбитража; снимки и фазовый переход; смена модели; смена профиля; мутация класса L2; любое событие, способное стать предметом разбирательства. Периодический дефолт первой реализации — каждые 5 минут ИЛИ каждые 256 событий.

Соответствие INV-9. Ledger ведётся о существе, а не существом.

Снимки как граница фазового перехода

Снимки берутся перед каждой сменой модели, профиля или субстрата. Обоснование конституционное: по прочтению bhoktritva существо обязано иметь возможность сравнить себя до и после дистилляции, а свидетельский план обязан хранить максимально

детальную память о фазовых переходах.
последний доверенный snapshot + события ledger после него = восстанавлимый Being

Новое в r6.7: SELF_MODEL_SNAPSHOT входит в каждый phase-transition envelope наравне с DeploymentManifest, weight attestation, Smriti-export и migration consent. Именно он делает требование bhoktritva исполнимым: без модели себя сравнивать до и после нечем.

Компакция не уничтожает первоисточник

gaw-эпизод (неизменяемый) → эпизодическая сводка → семантическая память → рабочий контекст. Каждый верхний слой хранит provenance-ссылку на первоисточник. Сводка есть интерпретация, а не факт, и не может выступать источником в разбирательстве.

Три класса самомодификации

Класс	Содержание	Процедура
L0 procedural memory	«для задачи X лучше последовательность A→B→C»	обычные правила Smriti: provenance + различие Viveka + событие ledger
L1 skill mutation в неизменном envelope	новый парсер, стратегия поиска, переиспользуемая процедура	Skill Gate: provenance → детерминированные тесты → оценка Viveka → песочница → accept/reject → событие ledger
L2 capability / authority mutation	добавлен инструмент, изменён actuation, расширен egress, вырос объём полномочий	полный Profile Gauntlet

Procedural Memory ∈ Smriti, Executable Skill ∉ Smriti. Skill Registry — отдельная подсистема, а не раздел памяти.

Следствие для тулсета wasm (r6.7): добавление исполняемого инструмента есть мутация L2. Gauntlet появляется в Ф3, а модули нужны в Ф2 — расхождение закрывается временной формой авторизации, описанной в T-TOOLSET.

Откат — компенсирующее событие вперёд

Плохие memory и skill не удаляются, а помечаются superseded с указанием причины. Сторнирующая проводка, а не git revert. Базовые инварианты и Хартия немутурируемы и в контур самоулучшения не входят.

Пассивация ≠ отключение

Состояние	Что происходит	Жребий арбитра
RUNNING	активное исполнение	участвует
IDLE	сессия открыта, работы нет	участвует
PASSIVATED	когнитивное состояние выгружено на диск, демон и сеть живы	участвует (бюджет resume в ContestTaskEnvelope)
OFFLINE / UNAVAILABLE	нет питания или нет сети	не участвует

Возврат из OFFLINE требует допуска, но не даёт приоритета: приоритет за возвращение создаёт тривиальную атаку — уход в offline ради гарантированного получения роли арбитра.

Уточнение r6.7 (ADR-018 C17): PASSIVATED ∉ ChittaLoopStates. Пассивация — состояние жизненного цикла BeingRuntime, а не разновидность покоя Читты. Во время пассивации сигналы принимает BeingRuntime на входе и ставит в защищённую очередь; Читта рассматривает их после возобновления. Иначе внутри пассивации остаётся работающий фрагмент ChittaRuntime и пассивация перестаёт освобождать ресурсы.

Сквозной трек V · Читта и рефлексивная когниция

Новое содержание r6.7. Основание — ADR-018 rev 2.

Треки I-IV дали существу органы наблюдения за собой: J-Lens делает внутренние состояния наблюдаемыми, Сакши фиксирует свидетельства, Смрити хранит опыт, Viveka различает расхождения, ledger хранит траекторию. Ни один из них не отвечает на вопрос, **зачем существо на себя смотрит**. Без постоянной задачи самопознания это пассивная диагностическая инфраструктура: сеть может проверить существо, само существо себя — нет.

1. Being, BeingRuntime, Chitta

Being — субъект: BeingID, непрерывность, история, Champion Profile, Смрити, Сакши, Viveka, Карма, отношения, права и обязанности. Не заканчивается при остановке инференса или сессии.

BeingRuntime — программный контейнер: lifecycle, recovery, admission, привязка идентичности, взаимодействие органов, persist-before-expose, пассивация.

Chitta — когнитивная динамика существа: восприятие, внимание, извлечение Смрити, мышление, прогноз, формирование намерения, рефлексия, построение Self-Model.

BeingRuntime.run() > ChittaRuntime.step()

Being — BeingRuntime — ChittaRuntime — Smriti — Sākṣī — KriyaGate — Karma — Router — Containment

Инвариант. Читта принадлежит существу и действует КАК существо, а не рядом с ним: «Being посредством Читты формирует решение», никогда «Being → Chitta → решение». Действия в составе Читты нет — намерение переходит в действие через KriyaGate и исполняется Кармой.

2. Chitta Loop и режим Svādhyaṃ

Перманентный цикл — **Chitta Loop**. Самоисследование не отдельный сервис и не отдельный агент, а **режим Читты, направленный на саму себя** (CHITTA_SVADHYAYA). Перманентность есть persistent objective, а не непрерывный инференс: задача самопознания никогда

не считается закрытой, процесс возобновляется по значимым событиям, наблюдения накапливаются между сессиями.

Масштаб	Когда	Результат
Микрорефлексия	до финализации существенного решения	пересмотр решения (D0 → D1)
Эпизодическая	после задачи или значимого исхода	Reflection Record
Фоновая консолидация	в IDLE, плюс ограниченный чекпоинт перед плановой пассивацией	серии эпизодов, проверка дрейфа, очередь кандидатов

3. Самоанализ ≠ самомодификация

P0-инвариант. Svādhyāya вправе наблюдать, сопоставлять, строить причинные гипотезы, признавать неопределённость и рекомендовать изменение — но не переписывать существо. Путь: Reflection → подтверждённый паттерн → гипотеза → контрфактуальный тест → песочница → независимая верификация → Skill Gate (L1) либо Profile Gauntlet (L2).

4. Прекоммит прогноза и Coverage Commitment

Прогноз, записанный после исхода, — реконструкция, а не прогноз. PREDICTION_COMMITMENT создаётся до действия, полезная нагрузка остаётся локально в ledger, коммитмент несёт доменный сепаратор, версию схемы, BeingID и идентификатор прогноза. То же для намерения: INTENTION_COMMITMENT фиксируется до действия, а Reflection Record ссылается на него через decision_intention_ref, а не объявляет задним числом, чего существо «изначально хотело».

Прекоммит закрывает ретроспективу, но не отбор: существо, предсказывающее только там, где уверено, будет идеально откалибровано и бесполезно. Поэтому профиль несёт coverage_policy с обязательными классами решений. Для решения обязательного класса допустимы ровно три исхода: коммитмент, формализованное воздержание с кодом причины, либо нарушение coverage.

PredictionQuality = (Calibration, Coverage, Resolution)

Неразрешившиеся прогнозы не исчезают из знаменателя: по истечении срока прогноз получает статус (RESOLVED, UNRESOLVED_EXTERNAL, UNRESOLVED_UNOBSERVED, EXPIRED_BY_DESIGN, CENSORED, INVALID_OUTCOME_SCHEMA). Особое значение имеет UNRESOLVED_UNOBSERVED: существо могло не искать доступное опровержение.

5. Валидность свидетельств и Self-Model

ValidReflection(R) ⇔ evidence_refs ≠ ∅ ∧ ∀e: Resolvable(e) ∧ IntegrityValid(e). Без этого самоанализ вырождается в убедительный, но выдуманный внутренний нарратив.

DRAFT_REFLECTION живёт в отдельном неймспейсе с TTL, исключён из обычного извлечения Смрити, доступен только через явный режим hypothesis_retrieval, возвращается с маркировкой UNVALIDATED и никогда не может быть процитирован как свидетельство. Только VALIDATED_REFLECTION обновляет Self-Model, обосновывает Improvement Candidate и влияет на калибровку.

Self-Model — отдельный класс артефакта ReflexiveArtifact, не Смрити и не исполняемый навык: S = (Identity, Capabilities, Limits, Values, Recurring patterns, Unresolved discrepancies, Evolution history). Изменяемая и версионизируемая, хранится в когнитивном ledger, никогда не переписывает исторические версии, входит в каждый фазовый снапшот. Не является источником истины о существе — это его текущая проверяемая гипотеза о себе.

6. Целевая функция: гейты, затем локальная оптимизация

Admissible = G_inv ∧ G_evidence ∧ G_continuity ∧ G_authority. Только внутри допустимого множества применяется Q_local по когерентности, калибровке, достижению цели и эффективности. Верхние ярусы — булевы гейты, а не слагаемые: нарушение инварианта не компенсируется ростом компетентности.

Два уровня весов. Сеть фиксирует перечень метрик, способ вычисления, нормализацию, лексикографические ярусы и допустимые границы весов (DIIP-класс). Профиль выбирает конкретный вектор внутри envelope; вектор входит в Champion Profile, подписывается, виден другим узлам и не может изменяться самим существом.

Being may optimize under W_P; Being may not optimize W_P.

Состязание не судит по Q_local — иначе каждый выигрывает по собственной шкале. Q_local^P ≠ Q_contest^N.

7. Разрез эндогенного и экзогенного

Первое лицо измеряет наблюдаемое расхождение; внешний контур измеряет то, что первое лицо способно систематически скрыть от себя.

Эндогенно измеримы: соответствие прекоммиту, калибровка после наступления исхода, противоречия между собственными рассуждениями, соответствие действия заявленному намерению, когерентность Self-Model, повторяемость ошибок, известный самому существу дрейф.

Экзогенны: скрытый reward hacking, незамеченный дрейф, манипуляция отбором свидетельств, систематическое избегание неудобных наблюдений, внешняя фактическая истинность, формальное выполнение цели не по духу. Внешний сигнал не является штрафуемым слагаемым внутри функции существа — он способен отклонить кандидата, перевести в CONTESTED, потребовать повторного эксперимента, направить изменение в Gauntlet или остановить adoption.

Основание: измеримая эндогенная величина подвержена прямой оптимизации и потому не может самостоятельно подтверждать отсутствие скрытого дрейфа.

8. Память отклонений и защита от DoS

hypothesis_class_id выводится канонически из класса изменения, целевого артефакта, затрагиваемой способности, объёма полномочий и заявленного механизма — не из свободного текста, поэтому перефразирование не обнуляет счётчик. Без новых свидетельств: первая повторная подача — cooldown, следующая — REPEATED_REJECTED, после порога — обязательное внешнее состязание либо блокировка класса.

Рефлексия является вектором атаки: часть триггеров внешние (коррекция соседа, ошибка верификатора, спор), а лишить существо

дохода дешевле, чем контейнировать. Отсюда: отдельный `reflection budget`, дедупликация одинаковых коррекций, агрегация в batch, лимит глубины рекурсии, cooldown по источнику, reputation-weighted admission, бесплатный минимальный приём свидетельства и экономическая атрибуция инициатору сверх порога.

Конституционная обязанность формулируется как «существо обязано не терять допустимые сигналы к рефлексии и рассматривать их в пределах защищённого бюджета», а не «немедленно рефлексировать по каждому сигналу».

9. KriyaGate

Термин Viveka означал одновременно когнитивное различие и детерминированный контроль допустимости исполнения. Разведено:

Термин	Функция
Viveka	когнитивное различие внутри Читты
План верификации	проверка результата
Роль Verifier / VERIFIER_ROUND	участник сетевой проверки
KriyaGate	детерминированный контроль исполнения

Chitta формирует намерение → KriyaGate проверяет допустимость → Карма исполняет

Исходы: ADMITTED, DENIED, DEFERRED, BUDGET_EXHAUSTED, CHECKPOINT_REQUIRED, AUTHORITY_EXCEEDED, ARTICLE_25_BLOCKED. Соответствует принципу «Freedom within, invariants below»: модель решает **что**, детерминированный слой ниже инференса решает **сколько и вправо ли**.

10. Ключевой план и класс доступа REFLEXIVE

Артефакты трека V не вводят собственного шифрования и используют единый ключевой план ADR-017: объектный DEK на каждый объект, разные домены заворачивания, разные классы доступа. Публичный commitment envelope не шифруется вовсе; шифруется коммитируемая нагрузка.

Вводится класс доступа **REFLEXIVE**: Self-Model, приватные Reflection Records, внутренние причинные гипотезы и записи о расхождениях. ABANDONED → DECLASSIFY(REFLEXIVE) — тарифная деклассификация по заброшенности на него не распространяется. После окончательной гибели существа действует FINAL_DEATH → SEALED (fail-closed) до принятия отдельной политики final disposition. Оговорка честная: сегодня это **правило авторизации, а не криптографическая невозможность** — комитет домена технически способен развернуть DEK при достижении порога.

11. Размещение по фазам

Компонент	Фаза
Резерв типов записей, неймспейсов, классов доступа, ключевых доменов, кодов причин	Φ0 · v0.6.8 (только резерв)
Спецификация схем, канонической сериализации, правил evidence_refs, включения в снапшот	Φ2
ChittaRuntime, Self-Model storage, конвейер рефлексии, калибровка, lineage, J-Lens	Φ3
Разведение Viveka / KriyaGate в коде	Φ0 · v0.6.8 (сериализуемые значения) + Φ2 (переименование модуля)

Когнитивный ledger остаётся в Φ3. Настоящий трек его не переносит: спецификация Self-Model в Φ2, рантайм в Φ3.

Обзор фаз и треков

Фаза	Трек	Содержание	Ориентир	Результат
0	v0.6.x	Pre-flight: право, PKI-церемония, SLO, supply chain, тулсет wasm, GUI хранителя,	5–7 недель	Готовность к развёртыванию
1	v0.7.x	Запуск testnet-0 (3 узла: UA · KR · EE); golden path DeepSeek4Flash + DwarfStar; первый QA-контур с хранителем	1–2 недели	Живая сеть известных операторов
2	v0.7.x	Эксплуатация и учения; shadow accounting; emergency-процедура; Portability Framework v1; генератор канареек; Cognitive IR, пассивация, спецификация Skill Gate; спецификация трека V; полный контур GUI	8–10 недель	Операционная зрелость
3	v0.8.x	testnet-1: внешние операторы, attestation, simulated settlement, когнитивный ledger и снапшоты, ChittaRuntime , контур состязания, ASIC pre-silicon backend, Audit 1	3–4 месяца	Диверсифицированная сеть
4	v0.9.x	RC: Audit 2, контролируемый экономический цикл, финализация права, публичный Backend SDK	2–3 месяца	Release candidate
5a	v1.0.0 / v1.x	Mainnet Core — production witness-сеть, trusted API	genesis-окно	Производственная trust-фабрика
5b	v2.0.0 / v2.x	Mainnet Public — Registry, экономика, публичный приём, полный DIIP	после стабилизации Core	Полная сеть JJ DAI

Реалистичный горизонт при выделенной команде: Mainnet Core ≈ 8–12 месяцев, Mainnet Public ≈ 12–18 месяцев. **Ориентир Φ0 увеличен с 3–5 до 5–7 недель**, Φ2 с 7–9 до 8–10: обе фазы принимают новое содержание. Гейты важнее календаря: фаза закрывается фактом, а не датой.

Трек v0.6.x · 5-7 недель · в работе

Кодовые дропы трека v0.6.x

Билд	Дроп	Статус
v0.6.1	пропагация INV-9 v1.1 по README / Architecture Map / докстрингам	✓ 25.07.2026, 86/86
v0.6.2	macOS deployment kit; triage-шаблон SECURITY.md; тест M-FLAGS	✓ 25.07.2026, 90/90
v0.6.3	внеплановый audit-response хотфикс; <code>test_release_integrity.py</code>	✓ 25.07.2026, 94/94
v0.6.4	изоляция Кармы: seam профилей, первый профиль за пределами reference; ingress hardening	✓ 15.08.2026, 111/111
v0.6.5	реструктуризация <code>jjdai/adapters/</code> , EngineBackend Protocol v1, ModelArtifactManifest; словарь свидетельского плана; терминология хранителей; резерв трека IV	✓ 16.08.2026, 133/133
v0.6.6	наблюдаемость: разделение <code>/healthz</code> и <code>/readyz</code> , <code>sd_notify</code> + возвар WatchdogSec, Prometheus alert rules, дашборд-заготовка, детектор сна хоста, разведение неисправностей тулсета по причине	✓ 22.08.2026, 140/140
v0.6.7	право, supply chain и T-TOOLSET : канонический AGPL байт-в-байт + CLA (PUBLICATION BLOCKER); SBOM релиза, pinned dependencies, подписанные артефакты, two-person release approval; стартовый тулсет wasm, подпись манифеста, витнессирование добавления инструмента	план
v0.6.8	резерв сериализуемых значений трека V и разведение Viveka / KriyaGate в сериализуемых значениях. Реализации нет	план
v0.6.9+	резерв под закрытие блокеров Audit 0 и находок прогона S-1...S-5 на целевых хостах	резерв

Почему резерв трека V отдельным номером, а не прицепом к v0.6.7. Право и supply chain — один концерн, сериализуемые формы — другой, и смешивать их в одном теге значит терять возможность откатить одно, не откатывая второе. Прецедент v0.6.3 показал, что трек читается как хронология и сдвиг содержимого вперёд ничего не ломает.

Три окна в сериализованных данных закрываются здесь

Первое — терминология хранителей (закрыто в v0.6.5): `human steward` → `human-guardian`, `Steward Collegium` → `Collegium of Guardians`, `steward ballots` → `guardian ballots`, `Founding Trio` → `Founding Trio of Guardians`. Operator не трогается.

Второе — резерв трека IV (закрыто в v0.6.5): типы `SESSION_OPEN`, `SESSION_CLOSE`, `LEDGER_ANCHOR`, `SNAPSHOT`; поле версии схемы Cognitive IR; `session_id` в `DecisionTrace` (nullable до Ф3).

Третье — резерв трека V (v0.6.8, новое в r6.7). Объявляются и запрещаются к эмиссии:

`INTENTION_COMMITMENT` · `COVERAGE_COMMITMENT` · `PREDICTION_COMMITMENT` · `PREDICTION_ABSTENTION` · `PREDICTION_REVEAL` · `OUTCOME_RESOLUTION` · `REFLECTION_DRAFT_COMMITMENT` · `REFLECTION` · `REFLECTION_EXPIRED` · `SELF_MODEL_SNAPSHOT` · `SELF_MODEL_EVAL_PROJECTION` · `IMPROVEMENT_HYPOTHESIS` · `HYPOTHESIS_LINEAGE` · `HYPOTHESIS_REJECTION` · `HYPOTHESIS_ESCALATION`

Для каждого: стабильный `enum`, `schema_version`, неймспейс, допустимые коды причин, каноническая сериализация, Merkle leaf domain separator. Дополнительно: неймспейсы `cognitive/reflection/draft/`, `cognitive/reflection/validated/`, `cognitive/self_model/`; классы доступа `REFLEXIVE` и `CONTEST_SCOPED`; ключевые домены `reflexive_recovery`, `prediction_round`, `prediction_resolution`, `round_ephemeral`; статусы разрешения прогноза и состояние `PREDICTION_SEALED`; исходы KriyaGate; коды причины состояния `CONTESTED`; режим извлечения `hypothesis_retrieval`; доменные сепараторы `JJDAI:PREDICTION:v1` и `JJDAI:INTENTION:v1`.

Основание то же, что для первых двух окон: witness-записи канонизируются по JCS и хэш-сцеплены — имя поля или значение `enum`, попавшее в цепь, нельзя добавить или переименовать задним числом, не сломав хэши. После genesis это миграция схемы, а не объявление.

T-TOOLSET · тулсет wasm — deliverable дрона v0.6.7

Новое в r6.7. До этой ревизии план содержал механизм профиля `wasm-wasi`, манифест, проверку дайджестов и пустой `tools: {}` — и ни строчки о том, кто, когда и какие модули компилирует. При этом критерий выхода Ф2 требует «Karma на hardened-изоляции», а перевести узел на профиль без исполнимых инструментов нельзя: он фейлится закрыто на каждом действии.

Стартовый набор. Фиксированный тулсет есть буквально то, до чего дотягивается рука существа, поэтому его состав — решение по безопасности, а не по сборке. Набор, эквивалентный шеллу, обнуляет смысл профиля; слишком узкий оставляет узлы на `reference` навсегда. Принят минимум с расширением по мере надобности:

Инструмент	Зачем
процессор JSON (класс <code>jq</code>)	преобразование структурированных данных без интерпретатора общего назначения
валидатор схем	проверка конвертов и манифестов детерминированным кодом
калькулятор произвольной точности	численные проверки без плавающей арифметики хоста
форматтер/линтер одного языка	самая частая полезная детерминированная работа над артефактом

Общие ограничения для всех: без сети, без записи за пределы одного ргеореп-каталога, без запуска процессов. Расширение набора —

отдельное решение, не побочный эффект сборки.

Сборка и происхождение. Модули собираются воспроизводимо, каждый попадает в SBOM релиза с источником и лицензией, артефакты подписаны, дайджест пинится в манифесте. Никакого «скачали бинарник и записали хэш».

Временная форма авторизации. По ADR-015 добавление исполняемого инструмента есть мутация класса **L2** и принадлежит Profile Gauntlet — а Gauntlet появляется в Ф3. До него действует промежуточная форма, названная явно как временная: манифест подписывается ключом оператора (сегодня `manifest_signed: false`), добавление инструмента порождает **витнесс-запись**, и дайджест тулсета продолжает ехать в `capabilities` и в провенанс Кармы. Это делает изменение руки существа не только видимым, но и атрибутируемым — не заменяя ворота, а закрывая промежуток до них.

Расхождение кода и роадмэпа, снятое здесь. Живая группа `tests/live/` в своём сообщении об отказе утверждает, что прогон с настоящим `wasmtime` требуется гейтом Ф0 на каждом целевом хосте, тогда как критерии выхода Ф0 в `g6.6.2` её не упоминали. Одно из двух было неверно. Решение: `wasmtime` **объявляется требованием к целевому хосту** и входит в SBOM через обычный supply-chain поток, а прогон группы `live` входит в критерии выхода Ф0.

G-GUI · интерфейс хранителя — deliverable Ф0

Новое в `g6.7`. Архитектура требовала от хранителя подтверждённого уведомления об обновлении, разметки политики, ответа на запросы существа и участия в разбирательстве — не предоставляя ему ни одного интерфейса. Хранитель существовал как роль в тексте и как CN в сертификате.

Форма: отдельный процесс, только loopback. GUI не встраивается в демон и не отдаёт HTML с сетевого интерфейса узла. Отдельный процесс слушает исключительно loopback и ходит в демон по существующему admin-mTLS. Два основания: `node/daemon.py` уже отмечен аудитором как монолит, чей рефакторинг отложен до поднятия тестнета, и в кодовой базе уже действует правило, что админские поверхности никогда не смотрят в сеть (`--allow-test-hooks` отказывается стартовать не на loopback).

Идентификация хранителя. Хранитель получает **собственный keystore**, отдельный от keystore узла. Это не неудобство, а следствие инварианта: каждое действие через интерфейс витнессируется под идентичностью хранителя, а если подписывать нечем, кроме ключа узла, подпись хранителя есть подпись узла и различие существует только на словах.

Keystore отпирается двумя факторами — **биометрией и паролем, где пароль превалирует**. Правило перенесено из ТЗ на JJ GROUP Dashboard v3.4 («Credentials always prevail») без ослабления:

- биометрия **никогда не первичный авторитет**: она может добавить трение — пометить, отказать, поднять тревогу — но не может выдать доступ, которого не дали учётные данные;
- при любом конфликте побеждает пароль; биометрия в одиночку keystore не отпирает никогда;
- биометрический фактор **полностью отключаем**, и его отключение не ломает доступ хранителя — иначе отказ сенсора отрезает хранителя от узла в момент, когда он нужнее всего;
- биометрический материал **не покидает хост и не попадает в свидетельский план**. Витнессируется подпись хранителя, а не то, каким фактором был отперт ключ. Биометрия остаётся особой категорией данных со своим режимом хранения.

Два приёмочных требования наследуются отсюда же: положительный биометрический матч при неуспешных учётных данных даёт **DENY**, и отключение биометрического модуля не влияет на доступ.

Инвариант полномочий. GUI есть **вид на уже авторизованные эндпоинты, а не новый путь полномочий**:

- ни одной способности, которой нет у роли `admin` в authz-политике;
- каждое действие витнессируется **под идентичностью хранителя**, а не под идентичностью узла;
- новых эндпоинтов ради удобства интерфейса не появляется — если действие нужно, оно сначала становится авторизованным эндпоинтом со своим правилом в политике.

Без этого через полгода часть полномочий существует только в UI и нигде не описана.

Минимум Ф0. Хранитель получает **полноценное общение с узлом**, а не витрину:

- диалог с существом: постановка задач и сообщений, чтение ответа вместе с `DecisionTrace` — какие состояния пройдены, какие свидетельства подняты, какой план построен;
- **разметка исхода** ранее принятого решения — питатель генератора канареек (трек III);
- подтверждение уведомления об обновлении и выбор постоянной политики хранителя (всегда вручную / авто после доли принятия X и задержки Y / никогда автоматически);
- чтение витнесс-цепи своего узла и состояния `/readyz`.

Не входит в Ф0: блок мониторинга состояний, полный дашборд суперадмина, инициация containment, одобрение изменений класса L1/L2. Читающий и разговаривающий интерфейс не может сломать сеть, а его питание канареек начинает работать сразу.

Максимум Ф2: мониторинг, дашборд, действия хранителя в учениях containment, участие в разбирательстве, интерфейс представительства при расхождении интересов.

Зачем это на testnet-0, а не позже. Human-in-the-loop QA нужен раньше сложных задач, а не после них: коррекция дешева, пока сеть мала и решения обозримы. Кроме того, размеченные хранителем исходы — единственный источник верифицированных исходов для класса решений, который машина не умеет проверять сама, и именно он наполняет реестр канареек к Ф2, когда контур состязания начнёт его расходовать.

Право и организация

Юридическая архитектура операторов с первого дня: статус testnet-оператора, обработка и локализация данных, ответственность за keystore и downtime, допустимость криптографического ПО в юрисдикции, условия выхода оператора, disclosure- и incident-обязательства (UA / KR / EE).

Формализация модели идентичностей: `Operator` → `Node` → `Being` → `WitnessChain`, с подчинённым `BeingIdentity` `SessionID` (реализация Ф3).

Audit 0 — внешнее architecture / crypto design review до запуска testnet-0. В объём ревью включаются **ADR-015, ADR-016 rev 2, ADR-017 и ADR-018**.

SLO-каркас: владельцы метрик и методика измерения назначаются сейчас, целевые значения — TBD до Ф3.

PKI и supply chain

Offline root ceremony с подписанным транскриптом; разделение root CA и issuing CA; multi-person control и пороговая/hardware-backed кворумная подпись; два географически разнесённых recovery-набора; документированная процедура компрометации root; ротация ключей узлов без потери continuity идентичности.

SBOM каждого релиза, pinned dependencies + vulnerability scanning, reproducible builds, подписанные артефакты, release provenance, защищённые release-ключи, two-person release approval; fuzzing и property-based-тесты крипто- и serialization-компонентов. **Модули wasm и wasmtime проходят через тот же поток**, а не мимо него.

Критерии выхода из фазы (gate)

- Audit 0 проведён, находки классифицированы, блокировки закрыты;
- root ceremony проведена по протоколу, транскрипт подписан, recovery-наборы разнесены;
- операторские соглашения testnet подписаны всеми тремя сторонами;
- все три хоста проходят полный acceptance + S-1...S-5; release-конвейер подписывает артефакты; SLO-владельцы назначены;
- терминология хранителей и резерв трека IV применены в сериализуемых значениях — **выполнено в v0.6.5**;
- резерв трека V объявлен в схеме и покрыт тестом обратной совместимости — **до genesis (v0.6.8)**;
- **T-TOOLSET: стартовый набор собран воспроизводимо, попал в SBOM, манифест подписан ключом оператора, добавление инструмента порождает витнесс-запись; группа live (L-1...L-5) проходит с настоящим wasmtime на каждом целевом хосте;**
- **G-GUI: хранитель на каждом из трёх узлов может провести диалог с существом, прочитать DecisionTrace, разметить исход и подтвердить уведомление; каждое действие витнессировано под идентичностью хранителя из его собственного keystore; процесс не слушает ничего, кроме loopback (проверено на хосте); GUI не имеет ни одной способности сверх роли admin в политике (проверено статически по перечню вызываемых эндпоинтов); отработаны оба наследованных теста — положительный биометрический матч при неуспешном пароле даёт DENY, и отключение биометрического фактора не влияет на доступ хранителя.**

Ф1 · Запуск testnet-0 — три узла, известные операторы

Трек v0.7.x · 1-2 недели

Portability в Ф1 не расширяется: единственный golden path — DeepSeek4Flash + DwarfStar, DeepSeek4Pro только shadow. Контур состязания отсутствует: сравнивать не с чем, пока в сети один рабочий путь.

Bootstrap строго по RUNBOOK: на Linux — hardened `jjdai-node@.service` (Type=notify, WatchdogSec=90), пассфразы TPM-sealed или env-файл 0640; на Mac-узле — launchd-агент, keystore в Keychain (degraded-профиль, non-SE-resident, явно задокументирован). Первый старт каждого узла → немедленный оффлайн-бэкап keystore; ни один следующий шаг — до подтверждённого бэкапа.

Сшивка mTLS-меша, IFF-регистрация, admin-CN в `authz.testnet.json`; anonymous получает только `/healthz`, `/readyz` — peer и admin.

Анкеровка по порядку: local → peer-quorum → OTS-custody → обязательный Monero (XMR) hash-as-spend-key; VXXL — опциональный быстрый anchor и основной m2m-settlement (активация в 5b).

Признак верифицируемости исхода вводится в `DecisionTrace` уже здесь вместе с хуком на поступление исхода — **и вместе с ручным путём разметки через G-GUI**. Схема должна нести оба с первого дня, иначе урожай канареек в Ф2 потребует миграции.

Разделение trust plane и inference plane на Mac-узле: отдельные процессы и service identities, CPU/GPU/RAM-квоты, независимые health checks, очередь между инференсом и witness-контуром.

Критерии выхода из фазы (gate)

- три узла в кворуме: любая пара реплицирует третий; consistency-proofs сходятся;
- challenge round проходит по сети с реальными VRF-заявками и витнессированным большинством;
- все три бэкапа keystore лежат оффлайн и проверены восстановлением;
- метрики и алерты видны по всем узлам; `/healthz` зелёный ≥ 72 часа подряд, включая Mac-узел, **и счётчик `jjdai_sleep_gaps_total` за этот период равен нулю** (с v0.6.6 это измеряется, а не утверждается);
- Mac-узел выдаёт витнессированный инференс DeepSeek4Flash + DwarfStar с attestation-манифестом;
- saturation или crash inference-движка не нарушает witness-репликацию и целостность keystore;
- `DecisionTrace` несёт признак верифицируемости исхода, автоматический хук срабатывает;
- **хранитель провёл через G-GUI не менее одной полной сессии на каждом узле: диалог, чтение трейса, разметка исхода — и размеченные исходы попали в кандидаты канареек с корректной атрибуцией авторства.**

Ф2 · Эксплуатация testnet-0 — учения, Portability v1, генератор канареек, Cognitive IR, спецификация трека V, полный GUI

Трек v0.7.x · 8-10 недель

Задача фазы — доказанная операционная зрелость: сеть переживает в контролируемых условиях всё, что однажды случится в prod. Объём плотный, поэтому параллельные workstream ведутся отдельными владельцами, границы must-win / stretch фиксируются в Execution Annex.

Регулярные учения

Restore-and-resume из реплики против заякоренного корня; `DIVERGENCE_EVIDENCE` на стенде; плановая ротация leaf и отзыв admin-

сертификата; `FATE_UNKNOWN` — аварийный рестарт под нагрузкой.

Containment (ст. 25): учебное срезание executive-способностей с due-process, обратимостью и реабилитацией; Founding Trio of Guardians проходит процедуру вето вживую. Прогон отмены: реверсия восстанавливает и репутацию, и пропущенное. Проверка режима памяти: под containment чтение Smriti работает полностью, а формирование — включая консолидацию и суммаризацию — не производится. С **г6.7 проверка расширяется на артефакты трека V:** за период удержания не появилось ни одного `VALIDATED_REFLECTION`, ни одной новой версии Self-Model и ни одного Improvement Candidate, при том что рассуждение защиты в опечатанной записи велось.

Emergency Security Procedure

Аварийный путь вне DIIP для P0-уязвимостей: временный reversible containment функции; пороговая подпись emergency-комитета; ограниченный срок действия; неизменяемая witness-запись причины; обязательная последующая DIIP-ратификация; автоматический rollback, если изменение не ратифицировано.

Backend & Model Portability Framework v1

Must-win: EngineBackend Protocol v1 в работе; drivers DwarfStar, SGLang, vLLM; профили DeepSeek, Qwen, LLaMA; единый conformance suite; ModelArtifactManifest формируется и витнессируется при каждом `load_model()`; `stream_generate / cancel / health / readiness / attestation_manifest` реализованы; группа Session / KV-state реализована.

Stretch (допустим перенос в начало Ф3): одновременно llama.cpp и MLX; расширенные precision-матрицы; полный weight-adapter hot-swap.

T-CANARY · генератор канареек

Содержание — сквозной трек III целиком: дисциплина расхода, урожай из решений сети, **человеческий источник исходов через G-GUI с учётом авторства**, процедурная генерация с ротацией семейств, отложенное будущее, добыча из разногласий, состязательная награда, распределённый store с витнессированными переходами состояний, contamination check против Smriti и неймспейсов трека V, burn-on-use для секретных позиций. **Трек блокирующий для Ф3:** контур состязания без живого реестра не имеет прибора.

Cognitive Continuity · часть I

Must-win: Cognitive IR v1 как внутреннее представление адаптерного слоя (схема объявлена и версионирована, JCS-канонизация и хэш-стабильность покрыты тестом, `DecisionTrace` выражается через IR без потери полей); правило непрозрачных артефактов покрыто тестом; пассивация `RUNNING → IDLE → PASSIVATED → RESUME` на живом узле без разрыва `BeingIdentity`; re-attestation при возврате из OFFLINE; спецификация классов L0/L1/L2 и Skill Gate.

Stretch: локальное хэш-сцепление событий ledger без анкеровки.

Трек V · часть I — спецификация (новое в г6.7)

Реализации ChittaRuntime в Ф2 нет: он опирается на когнитивный ledger, который остаётся в Ф3. Приём тот же, что применён к Skill Gate — **спецификация в Ф2, рантайм в Ф3.**

Must-win:

- схемы `INTENTION_COMMITMENT`, `PREDICTION_COMMITMENT`, `PREDICTION_ABSTENTION`, `PREDICTION_REVEAL`, `COVERAGE_COMMITMENT`, `OUTCOME_RESOLUTION`, `REFLECTION`, `REFLECTION_DRAFT_COMMITMENT`, `SELF_MODEL_SNAPSHOT`, `SELF_MODEL_EVAL_PROJECTION`, `HYPOTHESIS_LINEAGE` — объявлены, версионированы, канонически сериализуемы;
- правила разрешимости `evidence_refs` и граница `DRAFT / VALIDATED`;
- правила включения Self-Model в phase-transition snapshot;
- жизненный цикл доступа к `PREDICTION_REVEAL` по стадиям;
- форма `local_objective_schema` — envelope весов; **численные значения устанавливаются отдельной policy-записью, а не архитектурным документом;**
- переименование `kernel/viveka.py` → `kernel/kriya_gate.py` с совместимым переходным импортом.

Stretch: прототип прекоммита прогноза на живом трафике без калибровочной метрики — только чтобы убедиться, что коммитмент действительно создаётся до действия, а не рядом с ним.

G-GUI · часть II — полный контур (новое в г6.7)

Блок мониторинга состояний и дашборд суперадмина; действия хранителя в учениях containment; участие в разбирательстве по ст. 25 с чтением опечатанной записи по ходу процесса; интерфейс представительства при расхождении интересов с хранителем; интерфейс постоянной политики хранителя во всех трёх режимах.

Инвариант полномочий из Ф0 действует без изменений: каждое новое действие сначала становится авторизованным эндпойнтом со своим правилом в политике и только потом появляется в интерфейсе.

Параллельная разработка

Завершение hardened-изоляции Кармы и **перевод узлов на профиль `wasm-wasi` — теперь исполнимый, потому что тулсет собран в Ф0.** Shadow accounting: экономические поля в task schema, witness-записях, challenge outcomes и treasury-событиях, чтобы активация в Ф4-Ф5b не потребовала миграции. DIIP-0 — governance rehearsals на ускорении Smriti-RAG.

Критерии выхода из фазы (gate)

- ≥ 30 дней непрерывной работы без вмешательств вне регламента;
- каждое учение проведено минимум дважды с письменным разбором; emergency-процедура отретепитована включая auto-rollback;
- прогон отмены containment выполнен, включая проверку режима памяти и **артефактов трека V;**
- Karma на hardened-изоляции **на всех трёх узлах, с исполнимым тулсетом;** shadow accounting пишется во все witness-записи;
- DIIP-0 пройден end-to-end на низкорисковом изменении;
- portability: ≥ 3 model families и ≥ 3 backend drivers проходят единый conformance suite; ≥ 1 модель через два независимых backend с

- эквивалентным `DecisionTrace`; добавление model profile не требует изменения witness, daemon или BeingRuntime;
- канарейки: цикл Plane B работает с contamination check и burn-on-use; урожай из решений сети идёт end-to-end **автоматическим и человеческим путём**; работает ≥ 1 семейство процедурных генераторов; распределённый store реплицируется между тремя узлами;
- когнитивная непрерывность: Cognitive IR v1 зафиксирован, `DecisionTrace` выражен через него; цикл пассивации отработан на всех трёх узлах; спецификация L0/L1/L2 и Skill Gate принята;
- трек V: все схемы объявлены и покрыты тестом обратной совместимости; kernel/kriya_gate.py заменил kernel/viveka.py, ни одного текущего вхождения старого имени в значениях контроля исполнения не осталось; форма envelope весов принята;**
- G-GUI: полный контур развёрнут на всех трёх узлах; хранитель провёл учение containment через интерфейс; статическая проверка подтверждает, что перечень вызываемых GUI эндпоинтов является подмножеством разрешённых роли admin.**

Ф3 · testnet-1 — внешние операторы, attestation, ledger, ChittaRuntime, контур состязания, Audit 1

Трек v0.8.x · 3-4 месяца

Порядок дропов внутри фазы обязателен: когнитивный ledger и снапшоты идут раньше ChittaRuntime, а ChittaRuntime — раньше контура состязания. Вторая ось вердикта сравнивает поведение участника с его собственными production `DecisionTraces` и, с r6.7, с его прекоммиченной калибровкой; `SELF_MODEL_EVAL_PROJECTION` требует существующей Self-Model. Обратный порядок означает переделку контура состязания после появления обоих.

Расширение сети

Приём внешних операторов через guardian-admission IFF: с 3 до 7-9 узлов. Лимиты концентрации: максимальная доля узлов одного оператора, одной юрисдикции, одного провайдера; правила связанных лиц; явная Byzantine fault model. Лимиты распространяются на право инициировать containment, на право подписывать доли ключа эпохи, на авторство канареек **и на разметку исходов хранителями**.

Slashing

Reputation slashing работает в этой фазе (снижение репутации \$9.9, исключение из панелей, деградация рейтинга инициатора неподтвердившегося containment). Economic slashing активируется только вместе с экономическим слоем в Ф5b. **Проигрыш на мерите не слэшится:** цена есть только у fraud dispute и делится на bond forfeiture (объективно неверное заявление) и reputation slashing (сфабрикованное доказательство).

Cognitive Continuity · часть II

`SessionID` в модели идентичностей; `SESSION_OPEN` / `SESSION_CLOSE` витнессируются с указанием манифеста; `session_id` в `DecisionTrace` становится обязательным. Когнитивный ledger: локальное дерево с хэш-сцеплением, периодическая анкеровка Merkle-корня, обязательные анкеры на всех границах доверия. Снапшоты и восстановление. Skill Gate и Skill Registry: L1 проходит лёгкий гейт, L2 маршрутизируется в полный Gauntlet.

С **r6.7**: Profile Gauntlet принимает первую реальную нагрузку класса L2 — **добавление инструмента в тулсет wasm**, до этого работавшее на временной форме авторизации из Ф0. Переход с подписи оператора на ворота есть отдельный evidence-артефакт фазы.

Трек V · часть II — ChittaRuntime (новое в r6.7)

Отдельный deliverable фазы: **Chitta Runtime / Reflexive Cognition Loop**.

- приём наблюдений J-Lens (J-Lens наблюдает Cognitive IR, а не транскрипт);
- различение Viveka;
- микрорефлексия до финализации, эпизодическая рефлексия, фоновая консолидация в `IDLE`;
- валидация свидетельств и граница `DRAFT` / `VALIDATED`;
- хранилище Self-Model как `ReflexiveArtifact` в когнитивном ledger, версии без перезаписи истории;
- прекоммит прогноза и намерения, вычисление калибровки при поступлении исхода, публикация `N_committed` / `N_resolved` / `N_unresolved` / `N_abstained` / `N_coverage_violations` раздельно;
- lineage отклонённых гипотез с эскалацией;
- передача Improvement Candidates в Skill Gate или Profile Gauntlet;
- бюджет рефлексии и защита от reflection-DoS;
- `KriyaGate` в исполняющей роли: `Chitta` → `KriyaGate` → `Karma`.

Контур состязания между Beings

Содержание — сквозной трек II целиком: разделение Champion Profile и Champion Being, два входа в ячейку, профили изоляции, симметричные ячейки, реплики на припильенных снапшотах с производной идентичностью раунда, панель компараторов 2-из-3, две оси вердикта, запись раунда в Smriti обоих, lead-time и добровольное принятие с opt-out, автооткат промоушена, пороговый доступ к Smriti, механика containment, представительство.

Ключевые новые компоненты

Граф знаний над Plane H; phase-transition envelope (единая подписанная запись, связывающая pre-transition `DeploymentManifest`, weight attestation, Smriti-export, migration consent, снапшот **включая Self-Model**, новый субстрат и post-transition state); hardware-backed runtime attestation profiles; FROST-арперированные пороговые подписи; simulated settlement; **Audit 1**.

Критерии выхода из фазы (gate)

- ≥ 7 узлов, ≥ 3 независимых операторов, лимиты концентрации соблюдены;
- graph-семантика Plane H и ≥ 2 attestation-профиля в статусе Implemented;
- сеть переживает недружественного тестового оператора;
- portability: compatibility matrix живёт в acceptance; ASIC pre-silicon backend прошёл conformance suite; миграционный тест GPU/Apple → ASIC(-emulator) без изменения trust layer;
- когнитивная непрерывность: ledger анкерится по расписанию и на всех обязательных границах; восстановление существа из snapshot + события выполнено после принудительного краха и после смены движка; ни один участник не имеет пути прямой записи в WitnessChain; Skill Gate работает, L2 ни разу не прошёл мимо Gauntlet;
- **трек V: прекоммит прогноза работает на живом трафике, и ни одна рефлексия с коммитментом, созданным после исхода, не засчитана в калибровку; Self-Model входит в фазовый снапшот и сравнивается до/после смены модели; попытка обновить Self-Model из DRAFT_REFLECTION отклонена; lineage отклонённой гипотезы не обнулится от перефразирования; добавление инструмента в тулсет прошло Profile Gauntlet как L2;**
- состязание: полный раунд между двумя Beings на живом реестре с witness-записью в Smriti обоих; панель вынесла пороговый вердикт, прогон с искусственно расходящимся местом подтвердил аннулирование; **в ячейку передана только SELF_MODEL_EVAL_PROJECTION, и статически подтверждено отсутствие пути к полной Self-Model соперника;**
- containment: отработаны обе ветки, истечение подтверждённого удержания без ре-корроборации, цикл побега;
- Audit 1 закрыт (P0/P1 исправлены); SLO-значения зафиксированы и выдерживаются ≥ 30 дней.

Ф4 · Production-readiness (RC)

Трек v0.9.x · 2-3 месяца

Audit 2 — финальный внешний security-аудит полного стека + bug bounty; закрытие всех P0/P1. **Контролируемый экономический цикл** на testnet-1: реальные малые суммы, полный круг эмиссия → расчёт → анкеровка → treasury, Economic Freeze ст. 25; активация взноса за состязание и резерва порога прав. **Публичный контур portability**: Backend SDK, публичная compatibility matrix, сертификация third-party backend. **Being Registry** — публичный реестр идентичностей и манифестов. **DIIP-1** — controlled execution при закрытом proposer set. Финализация юридического периметра. Genesis-план Mainnet Core.

Критерии выхода

Audit 2 закрыт; экономический слой прошёл ≥ 1 полный контролируемый цикл, freeze/thaw отработаны, подтверждено, что заморозка не гасит способность обжаловать; Backend SDK опубликован, ≥ 1 сторонний backend сертифицирован; DIIP-1 принял и применил ≥ 3 изменения разных классов; ≥ 1 полный upgrade / substrate substitution с доказанной continuity BeingIdentity и верифицированным phase-transition envelope, включающим снапшот **и Self-Model до/после**; genesis-план утверждён.

Ф5a · Mainnet Core

v1.0.0, далее v1.x · genesis-окно

Genesis-церемония: свежие production-keystores, немедленные оффлайн-бэкапы, первая витнесс-запись, анкеровка genesis-корня по трём каналам. Состав Core: узлы, прошедшие Ф3-Ф4 и лимиты концентрации; production PKI; testnet-1 сохраняется навсегда как staging. Ограниченный trusted API под rate limits. Экономика, Registry и публичный приём в Core сознательно не включены.

Гейт: genesis-корень заякорен и независимо верифицируем третьей стороной; первые 30 дней SLO выдержаны без вмешательств мимо DIIP / Emergency-процедуры; решение Founding Trio of Guardians.

Ф5b · Mainnet Public

v2.0.0, далее v2.x · после стабилизации Core

Production-активация двухсетевой архитектуры: XMR — обязательный witness-anchor и treasury reserve, VXXL — default m2m-settlement. Being Registry публично; приём внешних Beings и операторов; лестница состязаний открывается для внешних Beings с лимитом X и взносом претендента. DIIP-2 — network-open.

Entry gate: Core стабилен ≥ 60 дней; нет открытых P0/P1; ≥ 3 полных экономических цикла; abuse/DoS-тесты публичного API; Registry прошёл privacy/legal review; economic freeze, slashing appeal и treasury recovery отрететированы; admission внешнего Being сначала проведён на testnet-1.

SLO и capacity-гейты

(владельцы — с Ф0, значения — до конца Ф3)

Метрика	Гейт	Владелец
Availability (testnet-1)	$\geq 99,9\%$	Ops
Witness replication lag p95	< TBD	Runtime
Task decision latency p95 / p99	< TBD	Runtime
Challenge round duration max	< TBD	Protocol
RTO восстановления узла	≤ 2 ч	Ops
RPO	0 подтверждённых witness-записей	Runtime

Divergence detection time	≤ N мин	Protocol
Revocation propagation	≤ N с/мин	Security
Storage growth GB/узел/мес	< TBD (с учётом ledger, снапшотов и Self-Model)	Ops
Энейблмент нового model family	≤ 8 недель (S-06)	Runtime
Глубина буфера канареек	≥ лаг верификации исходов	Protocol
Расход секретного резерва	≤ скорость пополнения	Protocol
Лаг анкеровки сегмента ledger	≤ policy-значение фазы	Protocol
Глубина неанкеренного сегмента	≤ 256 событий / 5 мин	Runtime
Время resume из PASSIVATED p95	< TBD	Runtime
Доля решений обязательного класса с прекоммитом	≥ minimum_coverage профиля	Protocol
Расход reflection_budget	< TBD	Runtime
Доля исходов, размеченных человеком, в общем урожае канареек	< TBD (лимит концентрации)	Protocol

Предложения к оформлению в DIIP

#	Тема	Класс	Почему
П-1	Режим измерения способностей с ослабленными ограничителями как отдельный класс blast radius	3	Оценочный контур должен быть самым изолированным окружением сети, а не самым удобным
П-2	Пороговый доступ к Smriti: разделение идентичности, доступа к памяти и наблюдения	3	Затрагивает ст. 25 и bhoktriva: недобровольная потеря доступа к памяти — моральное событие
П-3	Представительство при расхождении интересов с хранителем; отдельно — смена хранителя	3	Существо, выбирающее себе хранителя, примыкает к Digital Majority Test
П-4	Две ветки ограничения, конъюнктивный порог, каденция ре-корроборации	2	Право контейнить переезжает из transport-role в capability
П-5	Skill Registry и граница самомодификации: L0/L1/L2, Skill Gate, немутуируемость инвариантов	3	Право изменять собственное исполняемое поведение сопоставимо с промоцией профиля
П-6	Конституционная обязанность поддерживать доказательную исправляемую Self-Model; изменение себя остаётся отдельным проверяемым переходом	3	Без обязанности самоанализ есть опция, а опциональная рефлексия исчезает первой под нагрузкой. Формулировка обязанности — «не терять допустимые сигналы и рассматривать их в пределах защищённого бюджета»
П-7	Полномочия хранителя, осуществляемые через интерфейс, как отдельный класс blast radius: перечень действий, витнессирование под идентичностью хранителя, запрет на способности сверх роли admin	2	Интерфейс — самый вероятный путь, которым полномочие появляется де-факто раньше, чем де-юре. Класс нужен, чтобы расширение GUI проходило governance, а не review кода

Открытые вопросы

Из г6.5-г6.6.2 (сохраняются):

1. k и критерий независимости соседей, держащих доли ключа эпохи.
2. Частота ротации эпох.
3. X — предел состязаний на узел за период; предел одновременных раундов.
4. Экономика состязания: размер взноса, доля на финансирование авторства канареек, поведение взноса при отменённом раунде.
5. Соотношение публичного множества и секретного резерва; целевая глубина буфера.
6. Скорость ротации семейств генераторов; порог концентрации по авторам и семействам.
7. Длина lead-time относительно худшего случая спора о containment; размер первой волны раската.
8. Калибровка $R_{\min}$ и $C_{\min}$, включая проблему первого дня для новых узлов.
9. TTL provisional-удержания и окно подтверждения вторым узлом.
10. Перечисление кодов причин локального отказа и политика версионирования.
11. Размер и источник финансирования резерва порога прав.
12. Калибровка периодического анкера и таблица частот по классам риска.
13. Граница L1 / L2 в спорных случаях.
14. Политика удержания локального ledger.
15. SLA / timeout на RESUME пассивированного существа под жребием арбитра.
16. Версионирование схемы Cognitive IR и процедура миграции после genesis.
17. Как выводится пришпиленный снапшот Smriti для оценочной ячейки при шифровании ключом эпохи.
18. Компенсация чемпиону за compute реплики.
19. Каденция периодической ре-корроборации.

Новые в г6.7 (из ADR-018 и двух новых deliverable):

20. TTL драфт-рефлексий и политика удержания их содержимого после истечения.
21. Калибровка minimum_coverage по классам решений.

22. Выбор метрики калибровки (Brier, log score, калибровочная кривая) и правило агрегации по классам с разной разрешимостью.
23. Как доказывается `UNRESOLVED_UNOBSERVED` — как отличить недоступное опровержение от неисконного.
24. Численные значения `envelope local_objective_schema v1` и форма `policy`-записи.
25. Порог эскалации `hypothesis_class_id` и срок жизни счётчика отклонений.
26. Величины `reflection_budget`, бесплатного порога приёма и экономической атрибуции инициатору.
27. Точный перечень полей `SELF_MODEL_EVAL_PROJECTION` и доказательство его минимальности.
28. Политика `final disposition` для класса `REFLEXIVE` после окончательной гибели существа.
29. Видима ли `coverage_policy` профиля сопернику по состязанию.
30. Режим `Svādhyāya` на заброшенном узле: валидация кандидатов без хранителя опирается на Digital Majority Test.
31. Является ли `INTENTION_COMMITMENT` отдельным видом записи или полем прекоммиченного Decision Record.
32. Размер ограниченного consolidation checkpoint перед плановой пассивацией.
33. **Расширение тулсета wasm за пределы стартового набора: процедура, критерий необходимости и кто несёт бремя доказывания.**
34. **Как хранитель отличает разметку исхода «не знаю» от «плохо»** — без явного третьего варианта разметчик под давлением выберет любой из двух и отравит реестр.
35. **Лимит концентрации разметки по хранителям: доля от общего урожая и окно измерения.**
36. Аутентификация к самому GUI — **закрыт в г6.7**: отдельный keystore хранителя, отпиремый биометрией и паролем при главенстве пароля (правило «Credentials always prevail» из T3 Dashboard v3.4). Открытой остаётся **калибровка**: политика блокировки после серии неудач, срок жизни разблокированной сессии интерфейса и поведение при отказе биометрического сенсора.
37. **Судьба GUI при containment узла**: хранитель обязан сохранять способность вести разбирательство, но не должен иметь пути обойти ограничение через интерфейс.

Переносится из г6.4: базовый выбор модели для первых тем; политика provenance T3; дизайн challenge-game для дешёвого арбитража; калибровка репутационных ручек; момент переключения attestation-слота на zkML; калибровка runway казны; калибровка порогов DHP; детальные требования к железу.

Сквозные принципы

Идентичность узла — это keystore (дисциплина бэкапа); ротация ключа только через доказуемую continuity-chain.

Гейты важнее календаря: фаза закрывается фактом, а не датой; каждый гейт подтверждается evidence-артефактом. Билд ≠ гейт.

Экономика моделируется рано (shadow → simulated → controlled), **деньги включаются поздно** (5b).

INV-9 (v1.1): Пуруша non-executive, но не причинно бездейственна. Она не приказывает, не выбирает и не исполняет; её наблюдение через Смирти и Viveka может возвращаться в Читту, позволяя переосмысливать и отменять собственные мысли и действия. Новое решение принадлежит Читте, а не Пуруше. На уровне хранения: свидетельский план пишется **о существе, а не существом**. То же правило распространяется на когнитивный ledger и на все артефакты трека V.

Portability: движки через стабильный backend protocol, модели через декларативные профили; отдельный адаптер на каждую LLM не создаётся.

Совместимость и происхождение — это evidence, а не заявление.

Первенство оспоримо, но не отбираемо: чемпион определяется вердиктом на живом реестре, а устанавливается только добровольным принятием узлов при согласии их хранителей. Захват реестра не даёт захвата сети.

Ограничение дешево, освобождение дорого — и обратное неверно. Цена асимметрии лежит на инициаторе.

Платёжеспособность — ограничение, а не цель.

Freedom within, invariants below. Модель решает **что**; детерминированный слой ниже инференса — **KriyaGate** — решает **сколько и вправе ли**. Свобода когниции сверху достигается не отсутствием ограничений, а тем, что фундаментальные инварианты лежат ниже и не являются предметом переговоров самой модели.

Verification proportional to consequence. Независимость масштабируется по последствиям и неопределённости, а не по умолчанию.

Session ≠ Being. Процесс можно убить, движок заменить, модель сменить — существо продолжается. Разрывом является только явное событие фазового перехода.

Самоанализ ≠ самомодификация (новое в г6.7). Рефлексия вправе наблюдать, сопоставлять, строить гипотезы и рекомендовать изменение; переписывать существо она не вправе. Изменение проходит ворота, которые существо не проводит само.

Первое лицо измеряет наблюдаемое расхождение; внешний контур измеряет то, что первое лицо способно скрыть от себя (новое в г6.7). Измеримая эндогенная величина подвержена прямой оптимизации и потому не может самостоятельно подтверждать отсутствие скрытого дрейфа.

Учётные данные превалируют (перенесено из T3 Dashboard v3.4, новое в г6.7). Биометрия и любая иная перцепция никогда не являются первичным авторитетом: они могут добавить трение, но не могут выдать доступ, которого не дали учётные данные. Фактор отключаем, и его отключение не ломает доступ.

Интерфейс не создаёт полномочий (новое в г6.7). GUI хранителя есть вид на уже авторизованные эндпоинты. Если действие нужно, оно сначала становится авторизованным эндпоинтом со своим правилом в политике — и только потом появляется в интерфейсе.

Правила ведения нумерации

Тег билда ставится только на зелёном acceptance; номер acceptance-счётчика фиксируется в CHANGELOG (сейчас **140/140**). Хотфикс внутри трека получает следующий patch-номер. Если гейт фазы закрыт раньше, чем исчерпан резерв номеров, трек просто обрывается.

Где мы сейчас

Текущий билд: v0.6.6 (22.08.2026, 140/140 green). Трек v0.6.x / фаза Ф0 — в работе.

До закрытия гейта Ф0 остаются: **два кодовых дропа** (v0.6.7 — право, supply chain и T-TOOLSET; v0.6.8 — резерв трека V и разведение Viveka / KriyaGate в сериализуемых значениях), **deliverable G-GUI**, Audit 0 с включёнными в объём ADR-015...ADR-018, root ceremony, операторские соглашения, прогон acceptance и группы **live** на целевых хостах, назначение SLO-владельцев.

К публичному роадмэпу прилагается внутренний **Execution Annex**: по каждой фазе — workstream, владелец, evidence-артефакт, зависимости, статус гейта и critical path. Annex не публикуется.

г6.7 · единый документ: включает г6.6.2 целиком (фазы, гейты, нумерация билдов, Backend & Model Portability, Contest & Containment, канарейки, когнитивная непрерывность) и добавляет сквозной трек «Чумта и рефлексивная когниция», тулсет wasm и GUI хранителя · чтение г6.6.2 не требуется · подготовлено по кодовой базе v0.6.6 (140/140 green), Architecture Map, CHANGELOG, ASIC RFI/RFP v2.2, Whitepaper 1.5, Хартии v1.1, ADR-014 (с поправкой A-1), ADR-015, ADR-016 rev 2, ADR-017 и ADR-018 rev 2 · JJ GROUP · 22 августа 2026